

Clase 110 — Seguridad de APIs REST

Parte: 4 — Seguridad de aplicaciones web · Fuente: OWASP API Security Top 10 / Bug Bounty Bootcamp (Vickie Li) 🕒 Duración estimada: 110 min · Nivel: Avanzado

Objetivo

Auditar la seguridad de **APIs REST**, hoy el backend de casi toda aplicación moderna. Usaremos el **OWASP API Security Top 10** como marco, con foco en BOLA/IDOR a nivel de API, autorización rota a nivel de función y exposición excesiva de datos.

⚠️ **Ética:** solo en APIs propias/autorizadas. Acceder a datos de otros usuarios reales es un delito.

Resultados de aprendizaje

Al finalizar, el alumno podrá:

- 1. **Enumerar** endpoints y métodos de una API REST.
- 2. **Explotar** BOLA (Broken Object Level Authorization), el IDOR de las APIs.
- 3. **Detectar** Broken Function Level Authorization (acceso a acciones privilegiadas).
- 4. **Identificar** exposición excesiva de datos y mass assignment.
- 5. **Recomendar** autorización por objeto y por función, y filtrado de salida.

Temas

#	Tema	Por qué importa
1	OWASP API Security Top 10	Marco específico de APIs
2	Enumeración de endpoints	Superficie de la API
3	BOLA (API1)	La vulnerabilidad de API más común
4	Broken Function Level Auth (API5)	Escalada vertical
5	Excessive Data Exposure y mass assignment	Fugas y sobre-escritura
6	Rate limiting y abuso	Disponibilidad y coste
7	Defensa: authz granular	Cierre del fallo

Definiciones y características

- **API REST:** interfaz basada en HTTP y recursos con verbos (GET, POST, PUT, DELETE). Característica: cada endpoint necesita autorización propia.

- **BOLA**: acceder a objetos de otros usuarios manipulando su identificador. Característica: el IDOR de las APIs; la nº1 de OWASP API.
- **Broken Function Level Authorization**: usar funciones para las que no se tiene rol. Característica: escalada vertical vía endpoints admin.
- **Excessive Data Exposure**: la API devuelve más campos de los necesarios y el cliente filtra. Característica: fuga de datos sensibles.
- **Mass assignment**: enviar campos extra que el backend asigna sin filtrar (`isAdmin:true`). Característica: sobre-escritura de propiedades.
- **Documentación (Swagger/OpenAPI)**: especificación de la API. Característica: gran fuente de endpoints al auditar.

Herramientas y preparación

- **Postman** o **Burp** para construir peticiones a la API.
- **crAPI** o **VAmPI** (APIs deliberadamente vulnerables) y **Juice Shop**.
- Especificación **OpenAPI/Swagger** si está disponible.

```
# crAPI: API vulnerable de OWASP para practicar
git clone https://github.com/OWASP/crAPI && cd crAPI && docker compose up
```

Laboratorio guiado

 Solo en APIs propias/lab.

1. Descubre endpoints desde el Swagger/OpenAPI o analizando el JS del cliente.
2. Auténticate como usuario A y localiza `GET /api/users/{id}/orders` .
3. Cambia `{id}` al de otro usuario y comprueba **BOLA**.
4. Prueba **Broken Function Level Authorization**: llama a un endpoint admin (`/api/admin/ ...`) con tu token normal.
5. Inspecciona respuestas por **exposición excesiva**: ¿devuelve hashes, emails, roles no necesarios?
6. Prueba **mass assignment**: añade `"role":"admin"` o `"verified":true` en un PUT de perfil.
7. Evalúa el **rate limiting** enviando muchas peticiones y documenta el abuso posible.

Ejercicios

1. Enumera los endpoints de crAPI y agrúpalos por sensibilidad.
2. Explota un BOLA y documenta el dato de otro usuario obtenido.
3. Encuentra un endpoint sin control de función y escálalo.
4. Detecta exposición excesiva comparando lo mostrado en la UI vs. la respuesta cruda.
5. Realiza un mass assignment que eleve privilegios en el lab.
6. Diseña la autorización correcta por objeto y por función.

Reto verificable

En crAPI (o VAmPI), consigue **acceso a datos de otro usuario vía BOLA** y una **escalada por mass assignment**, documentando ambos. **Criterio de aceptación**: entregas las peticiones, la evidencia de acceso

no autorizado y elevación de privilegio, y la defensa (comprobar propiedad del objeto, allowlist de campos, authz por función).

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
Cambiar el ID da 403	Hay authz por objeto; busca otro endpoint
Endpoint admin devuelve 401	Requiere rol; prueba mass assignment para obtenerlo
Mass assignment ignorado	El backend filtra campos; documenta la fortaleza
No encuentro endpoints	Revisa Swagger y el JS del cliente
Rate limit corta las pruebas	Baja el ritmo; documenta el límite

Preguntas frecuentes

? ¿BOLA e IDOR son lo mismo? Conceptualmente sí; BOLA es el término de OWASP API para el IDOR a nivel de objeto en APIs.

? ¿Por qué la exposición excesiva es un problema si la UI no lo muestra? Porque el dato viaja al cliente y cualquiera puede leer la respuesta cruda; el filtrado en el cliente no protege nada.

? ¿Cómo evito mass assignment? Usa allowlists de campos aceptados (DTOs), nunca vincules el body directamente al modelo de datos.

Referencias

- OWASP API Security Top 10: <https://owasp.org/API-Security/>
- Li, *Bug Bounty Bootcamp*, sección de APIs.
- crAPI: <https://github.com/OWASP/crAPI>
- OWASP WSTG — API Testing.

Siguiente clase

Clase 111 - Seguridad de APIs GraphQL